Föreläsning 1

Abstrakta datatyper, listor och effektivitet

Föreläsning 1

- Datastrukturer
  - **A**bstrakta **D**ata**T**yper
  - ADT Lista
  - Lista och **J**ava **C**ollections **F**ramework (ArrayList)
  - Lista implementerad med en array

- Analys av algoritmers effektivitet och Big-O

Algoritmer och datastrukturer

- En algoritm är ett begränsat antal instruktioner/steg för att lösa en uppgift, som från givna indata med säkerhet leder till korrekta utdata.
- En datastruktur är en struktur som organiserar data
- Ett elementärt exempel är en array
- Val av datastruktur ska göras så att vi effektivt kan lagra, organisera och processa data
- För vissa problem är val av rätt datastruktur mer än halva lösningen!

Abstrakta datatyper ADT

En abstrakt datatyp definierar operationerna vi kan utföra på de data den skall lagra.

Den definierar inte implementationen.

I ett objektorienterat språk implementerar man gärna en ADT som en klass men det går också att implementera en ADT i exempelvis C.

Ex på ADT: lista, stack, kö

Kan implementeras med en array eller en länkad lista som intern datastruktur.

Observera att detta skiljer sig från hur begreppet användes i C-boken!

ADT Lista

Grundprinciper:

I en lista har varje element en position eller ett index.

Vi kan nå elementen i godtycklig ordning och sätta in eller ta bort element på godtycklig plats.

Precis som för alla ADT varierar det exakt vilka operationer man har med i definitionen. Nedan är ett minimum av operationer:

- size()
- get(index)
- add(index, element)
- remove(index)

Dessutom brukar man behöva någon form av create() eller initiate() beroende på implementation (i java en konstruktor).

JCF

Java samlar avancerade datastrukturer i **J**ava **C**ollection **F**ramework (alla inteface + klasser finns i **java.util**)

Här finns implementationer av ADT Lista:

- Klassen ArrayList implementerar en lista mha en array
- Klassen LinkedList implementerar en lista mha en länkad lista

För att det ska gå att byta dessa enkelt implementerar de båda interface:t List och kan därmed båda behandlas såsom ett objekt av typen List.

ArrayList

Använder en array för att lagra elementen i listan:

+ enkelt och effektivt att nå godtyckligt element via index

- tar upp onödigt minne då arrayen inte är full

- kostsamt när en ny array måste allokeras och alla element flyttas över då den gamla arrayen blivit full

- kostsamt då många element måste flyttas när man sätter in eller tar bort ett element mitt i listan

ArrayList JCF

java.lang.Object

java.util.AbstractCollection<E>

java.util.AbstractList<E>

java.util.ArrayList<E>

All Implemented Interfaces:

Serializable, Cloneable, Iterable<E>, Collection<E>, List<E>, RandomAccess

Class Arraylist<E>

Arraylist finns som generic vilket betyder att vi kan välja när vi skapar en arraylist vad det ska gå att lagra i denna:

```
List<String> myList = new ArrayList<>();
myList.add("hej");
myList.add("på");
myList.add("dig");
```

Vill vi lagra en primitiv datatyp måste vi använda motsvarande wrapper klass:

```
List<Integer> myList = new ArrayList<>();
myList.add(3);
```

Vi kan också använda en non-generic variant som då lagrar element av typen Object vilket är alla klasser i Java.

Detta är mycket sämre då vi inte får någon typchecking utan vi måste veta vilken typ av objekt vi tar ut och casta om det :

```
List myList = new ArrayList();
myList.add("Hej");
String s = (String)myList.get(0);
```

ArrayList – några metoder

boolean add(E e)

Appends the specified element to the end of this list.

void add(int index, E element)

Inserts the specified element at the specified position in this list.

E get(int index)

Returns the element at the specified position in this list.

int indexOf(Object o)

Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.

E remove(int index)

Removes the element at the specified position in this list.

E set(int index, E element)

Replaces the element at the specified position in this list with the specified element.

int size()

Returns the number of elements in this list.

Uppgift

Skriv en static-metod som returnerar hur många gånger en viss sträng förekommer i en List<String>

```
public static int count(List<String> list, String s){

}
```

Eftersom en ArrayList implementerar List kan man skicka en sådan till denna metod.

Lösningsförslag

```
public static int count(List<String> list, String s){
    int nrOf=0;
    for(String sl:list){
        if(s.equals(sl)) nrOf++;
    }
    return nrOf;
}
```

Implementera en lista med en array

```
package imparraylist;

import java.util.Arrays;

public class NArrayList<E> {

    private E[] data;
    private int nrElements;
    private int maxSize;

    public NArrayList(){
        nrElements = 0;
        maxSize = 10;
        data = (E[]) new Object[maxSize];
    }
 …
```

```
  public boolean add(E element){
      if(nrElements==maxSize)
          reallocate();
      data[nrElements++]=element;
      return true;
  }

  public E get(int index){
      if(0<=index && index < nrElements)
          return data[index];
      throw new ArrayIndexOutOfBoundsException(index);
  }

  private void reallocate(){
      maxSize*=2;
      data=Arrays.copyOf(data,maxSize);
  }
}
```

add(int index, E entry)

```
public void add(int index, E element){
    if(0<=index && index <= nrElements){
        if(nrElements==maxSize)
            reallocate();
        for(int i=nrElements;i>index;i--)
            data[i]=data[i-1];
        data[index]=element;
        nrElements++;
        return;
    }
    throw new ArrayIndexOutOfBoundsException(index);
}
```

Analys av algoritmers effektivitet

- Varför?
- Vad? - T(n) och O(f(n))
- Formell definition av Ordo (Big-O)
- Empirisk analys
- Vanliga tillväxthastigheter
- Analys av vår lista implementerad med en array

Varför analysera?

Behöver vi verkligen analysera algoritmer med dagens och morgondagens snabba datorer?

- Om tiden för en algoritm växer som n^2 kommer en 100 gånger så snabb dator bara att hinna med 10 gånger så stort problem.
- Om ett problem växer som 2^n är n=100 i princip olösligt ($2^{100} \approx 10^{30}$).
  - Ponera att vi kan hantera problem upp till n=40 ($2^{40} \approx 10^{12}$). Vi lyckas nu göra datorn 100 gånger snabbare. Då kan vi hantera n=47 ($2^{47} \approx 10^{14}$).

Kan man inte bara testa algoritmerna?

Förvisso en mycket bra ide som man inte bör glömma bort. Den har några problem (och en del fördelar):

- Man måste koda algoritmen (och göra det bra/rättvist)
- Vilka indata ska vi använda? typiska/slumpmässiga/extrema
- Med fel algoritm tar problemet för lång tid att testa
- Det gäller att tänka på overhead om vi använder små dataset.

Vad ska vi analysera? T(n) och O(f(n))

- När vi ska analysera en algoritm är vi intresserade av hur problemet som algoritmen ska lösa växer när problemets storlek växer

- Problemets storlek kan vara mängden data i ett dataset eller antalet input eller antalet värden vi vill räkna ut eller…
  - Storheten som växer betecknar vi med bokstaven *n* (om problemet kan växa i två oberoende dimensioner betecknar vi dessa *n* och *m*)

- Oftast är vi intresserade av hur tiden det tar att lösa problemet växer med *n* men det kan också vara hur minneskraven växer vi intresserar oss av.

- Den faktiska tiden det tar är svårmätt (overhead), hårdvaruberoende och även operativsystemberoende och därför inte så intressant. Istället är det intressanta hur många gånger enkla satser (ej beroende av *n*) exekveras som funktion av *n*. Denna funktion benämns komplexitetsfunktionen och betecknas T(*n*).

- Det händer att T(*n*) används för att beteckna tiden men det gör inte så stor skillnad då vi oftast inte är intresserade av den exakta funktionen utan bara hur den växer för mycket stora *n*. Vi säger att $T(n)=4n^2+2n$ är $O(n^2)$ (ordo-storleksordning).

Linjär tillväxt – O(*n*)

```
public static int search(int[] x, int target) {
  for(int i=0; i < x.length; i++) {
    if (x[i]==target)
      return i;
  }
  return -1; // target not found
}

//Exempel 2.4
```

O($n \times m$)

```
public static boolean areDifferent(int[] x, int[] y)
{
  for(int i=0; i < x.length; i++) {
    if (search(y, x[i]) != -1)
      return false;
  }
  return true;
}
//Exempel 2.5
```

Kvadratisk tillväxt O(n^2)

```
public static boolean areUnique(int[] x) {
  for(int i=0; i < x.length; i++) {
    for(int j=0; j < x.length; j++) {
      if (i != j && x[i] == x[j])
        return false;
    }
  }
  return true;
}
```

//Exempel 2.6

O(log *n*)

```
for(i=1; i < x.length; i *= 2) {
    // Do something with x[i]
}
```

$i = 1, 2, 4, \ldots, 2^{k-1} < x.length \leq 2^k$, där k är antal gånger loopen exekverar

$\log 2^{k-1} < \log(x.length) \leq \log 2^k$ (log betyder $\log_2$)

$k-1 < \log(x.length) \leq k$

Alltså får vi O(log *n*)

Formell definition av Ordo (Big-O)

- $T(n) = O(f(n))$ omm det existerar en positiv konstant c och ett heltal n_0 sådant att för alla $n > n_0$ gäller att $cf(n) \geq T(n)$
- $cf(n)$ är alltså en övre gräns för $T(n)$ för stora n
- Vi vill då hitta en funktion f(n) som växer så långsamt som möjligt men ändå uppfyller definitionen av ordo

Exempel - Ordo enligt definition

```
for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
      Simple Statement 1
      Simple Statement 2
  }
}
for (int i = 0; i < n; i++) {
   Simple Statement 3
   Simple Statement 4
   Simple Statement 5
   Simple Statement 6
   Simple Statement 7
}
Simple Statement 8
Simple Statement 9
. . .
Simple Statement 32
```

$T(n) = 2n^2 + 5n + 25$

$T(n) = 2n^2 + 5n + 25$

Vi väljer $f(n) = n^2$

vill hitta c och n_0 så att:

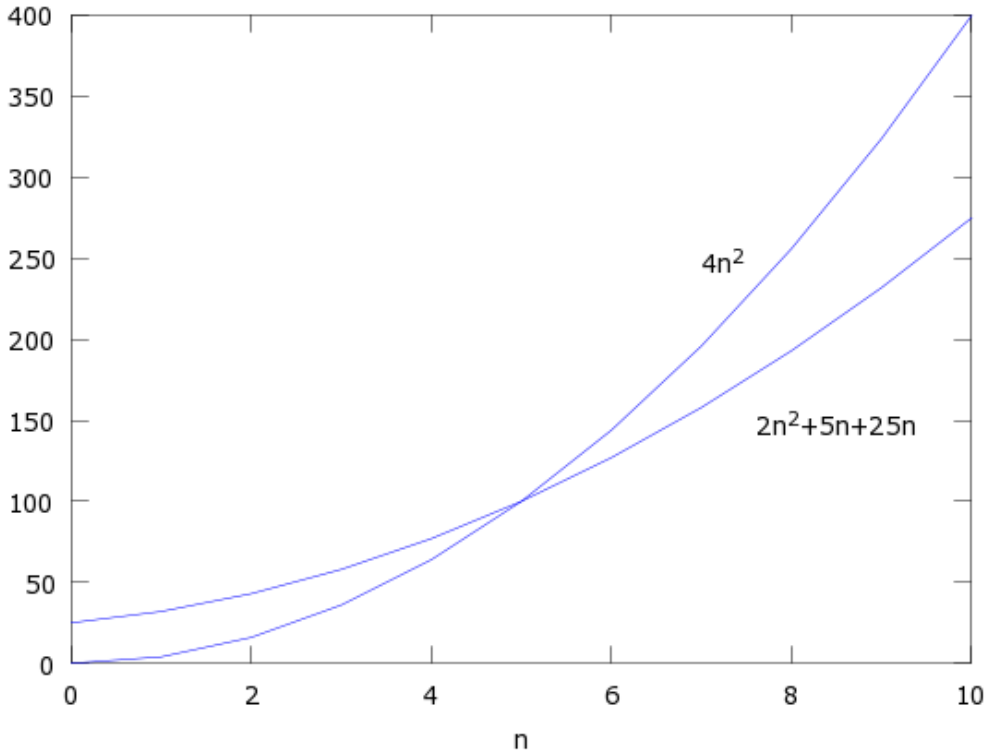
$2n^2 + 5n + 25 \leq c\, n^2$ för alla $n > n_0$.

Låt oss välja $n = 5$ och lösa ut c i motsvarande likhet:

$50 + 25 + 25 = 25c$

ger $c = 4$

Alltså: $T(n) = O(n^2)$ vilket kan visas med $n_0 = 5$ och $c = 4$.



Exempel 2

```
for (int i = 0; i < n; i++) {
  for (int j = i + 1; j < n; j++) {
      3 simple statements
  }
}
```

$$
\begin{aligned}
T(n) &= 3(n-1) + 3(n-2) + \ldots + 3 = \\
&= 3(n-1+n-2+n-3+\ldots+1) = \\
&= 3(1+2+3+\ldots+n-1) = \\
&= 3(n-1)(1+n-1)/2 = \\
&= 1.5n(n-1) = \\
&= 1.5n^2 - 1.5n = O(n^2) \text{ (kan visas med } n_0 = 1,\ c = 1.5)
\end{aligned}
$$

Exempel 3 - empirisk analys

```
int r=0,n=10;
    for(int i=1;i<=n-1;i++)
        for(int j=i+1;j<=n;j++)
            for(int k=1;k<=j;k++)
                r++;//enkel sats
```

n	0	1	2	3	4	5	6	7	8	9	10
T(*n*)	0	0	2	8	20	40	70	112	168	240	330

Låt ett matematikprogram anpassa en tredjegradare:

$T(n) = 0{,}3333n^3 - 0{,}3333n = O(n^3)$

Anpassning i matlab/octave

```
n=[0:10]
T=[0 0 2 8 20 40 70 112 168 240 330]
polyfit(n,T,3)
```

pröva också

polyfit(n,T,4) och polyfit(n,T,5)

Blev det någon skillnad?

Tips: Kör kommandot: format rational – så får du bråkform i matlab

Vanliga tillväxthastigheter i ökande ordning

Big-O	Name
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Log-linear
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential
$O(n!)$	Factorial

Låt oss anta att den enkla satsen tar 10^{-9} s

n	$T(n) = \lg n$	$T(n) = n$	$T(n) = n \log n$	$T(n) = n^2$	$T(n) = 2^n$	$T(n) = n!$
10	0.003µs	0.01µs	0.033µs	0.1µs	1µ	3.63ms
20	0.004µs	0.02µs	0.086µs	0.4µs	1ms	77.1år
30	0.005µs	0.03µs	0.147µs	0.9µs	1s	$8.4 \cdot 10^{15}$år
40	0.005µs	0.04µs	0.213µs	1.6µs	18.3min	
50	0.006µs	0.05µs	0.282µs	2.5µs	13dygn	
100	0.007µs	0.10µs	0.644µs	10µs	$4 \cdot 10^{13}$år	
1000	0.010µs	1.00µs	9.966µs	1ms		
10000	0.013µs	10µs	130µs	100ms		
100000	0.017µs	0.10ms	1.67ms	10s		
1000000	0.020µs	1mss	19.93ms	16.7min		
10000000	0.023µs	0.01s	0.23s	1.16dygn		
100000000	0.027µs	0.10s	2.66s	115.7dygn		
1000000000	0.030µs	1s	29.90s	31.7år		

Analys av vår lista implementerad med en array

n – antal element i listan, T – antal enkla satser

set(index), get(index) – O(1)

add(E) – O(1) utan anrop till reallocate();

– O(n) med anrop då vi måste flytta n element

– O(1) i genomsnitt eftersom vi dubblar storleken!

Antag att vi fyller en tom lista som startar med storleken m

add(index, E), remove(index) – O(n)

Antal add	Antal kopieringar
≤m	0
≤2m	m
≤4m	3m
≤8m	7m
≤16m	15m